Django REST Framework Study Guide

Backend learning guide for the travel blog project

Generated from the project files in the workspace

This guide is written for a complete beginner. It explains the code, the concepts, and the request lifecycle in plain English.

Table of Contents

1. Project overview

What this backend is, what each folder does, and how requests move through Django.

This project is a Django REST Framework backend for a travel blog. It exposes API endpoints for users, posts, and destinations. The backend accepts HTTP requests, routes them to Python code, talks to the database, and sends JSON back to the client.

The most important idea is that Django turns Python code into web behavior. You write Python classes and functions, and Django handles the connection to the web server, URL matching, database operations, and responses.

Core idea

Think of Django as a traffic controller. It receives requests, decides which Python function should handle them, and returns a response.

The project has three main apps: users, posts, and travel. Each app groups related features. The config package holds settings and routing rules.

The request lifecycle is: Browser/Postman → URL → Django URL dispatcher → View → Serializer → Model → Database → Serializer → JSON response.

```
Browser/Postman
   ↓
urls.py
   ↓
View
   ↓
Serializer
   ↓
Model
   ↓
Database
   ↑
Serializer
   ↑
JSON Response
```

Beginner definition

A view is the place where Django decides what to do for a request. A serializer is the translator between Python objects and JSON. A model is the Python representation of a database table.

The app folders are organized so that each feature has its own models, views, serializers, and URLs. This makes the code easier to understand and maintain.

2. Core startup files

manage.py, settings, URLs, and the WSGI/ASGI entry points.

The file manage.py is the command-line entry point for the Django project. When you run python manage.py runserver, Django uses this file to start the project and load configuration.

```
#!/usr/bin/env python
"""Django's command-line utility for administrative tasks."""
import os
import sys


def main():
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')
    try:
```

```
        from django.core.management import execute_from_command_line
    except ImportError as exc:
        raise ImportError(
            "Couldn't import Django. Are you sure it's installed and "
            "available on your PYTHONPATH environment variable? Did you "
            "forget to activate a virtual environment?"
        ) from exc
    execute_from_command_line(sys.argv)


if __name__ == '__main__':
    main()
```

Line-by-line explanation:

- The shebang tells Linux which Python interpreter to use when the script is executed directly.
- The docstring explains the purpose of the file.
- import os and import sys bring in Python modules for environment variables and command-line arguments.
- The main function is the entry point for Django management commands.
- os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings") sets the settings module so Django knows where configuration lives.
- execute_from_command_line(sys.argv) runs the command the user typed, such as runserver, migrate, or makemigrations.
- The if __name__ == "__main__": block ensures that main() runs only when the file is run directly, not when it is imported.

The settings file is the brain of the project. It contains database info, installed apps, middleware, authentication settings, allowed hosts, media folders, and DRF configuration.

```
BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = 'django-insecure-...'
DEBUG = True
ALLOWED_HOSTS = ['127.0.0.1', 'localhost']
```

- BASE_DIR points to the root of the project so paths can be built reliably.
- SECRET_KEY is a private string used by Django for security-related hashing; it should not be exposed in production.
- DEBUG=True means Django gives detailed error pages during development, which is convenient but unsafe in production.
- ALLOWED_HOSTS limits which domain names can reach the server.

The config/urls.py file is the top-level URL router. It sends requests beginning with /api/ to the users, posts, and travel apps.

```
urlpatterns = [
    path("admin/", admin.site.urls),
    path("api/", include("users.urls")),
    path("api/", include("posts.urls")),
    path("api/", include("travel.urls")),
]
```

- path("admin/", admin.site.urls) exposes the built-in Django admin dashboard.
- include("users.urls") adds the URL routes from the users app.
- The prefix /api/ makes the API endpoints easy to distinguish from the Django admin.

- If DEBUG is True, Django appends media serving rules to the URL patterns.

3. The users app

Authentication, OTP login flow, activity logging, and serializers.

The users app is the authentication and identity layer of the project. It handles registration, login, OTP verification, logout, and the activity log.

Concept: a serializer is a translator between incoming JSON and Python objects. It also validates user input. In this project, the serializers check usernames, emails, passwords, and OTPs before anything is saved.

The model file defines two database tables: OTP and ActivityLog.

```
class OTP(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE, related_name="otps")
    otp_code = models.CharField(max_length=6)
    created_at = models.DateTimeField(auto_now_add=True)
    expires_at = models.DateTimeField()
```

- The class OTP inherits from models.Model, which means it becomes a Django model and maps to a database table.
- user is a foreign key to Django's built-in User model. It means many OTP records can belong to one user.
- otp_code stores a six-digit one-time password.
- created_at is automatically filled when the row is created.
- expires_at stores the deadline for the OTP.

The save method overrides the default save logic. It sets the expiration time if the developer did not provide one.

```
def save(self, *args, **kwargs):
    if not self.expires_at:
        self.expires_at = timezone.now() + timedelta(minutes=5)
    super().save(*args, **kwargs)
```

- self refers to the current OTP instance.
- timezone.now() returns the current time.
- timedelta(minutes=5) means adding five minutes.
- super().save(...) calls the parent class implementation so Django can actually write the record to the database.

The serializer file uses DRF serializers to validate and transform user data.

```
class RegisterSerializer(serializers.ModelSerializer):
    password = serializers.CharField(write_only=True, min_length=8)

    class Meta:
        model = User
        fields = ['id', 'username', 'email', 'password']
```

- ModelSerializer is a DRF shortcut that automatically builds serializer fields from a Django model.
- write_only=True means the password is accepted for input but not returned in the response.
- min_length=8 enforces a minimum password length.
- The Meta class tells DRF which model and fields to use.

The validate_username and validate_email methods are custom validation hooks. They prevent duplicate usernames or emails.

```
def validate_username(self, value):
    if User.objects.filter(username=value).exists():
        raise serializers.ValidationError("A user with that username already exists.")
    return value
```

The create method uses Django's built-in create_user helper, which hashes the password correctly.

```
def create(self, validated_data):
    user = User.objects.create_user(...)
    return user
```

The LoginSerializer performs the login flow. It authenticates the user, creates an OTP, and returns a payload with the OTP details.

```
user = authenticate(username=user_obj.username, password=attrs['password'])
OTP.objects.filter(user=user).delete()
otp_obj = OTP.objects.create(user=user, otp_code=otp_code)
```

- authenticate checks the supplied username and password against Django user credentials.
- OTP.objects.filter(user=user).delete() removes old OTPs so users do not keep multiple active codes.
- OTP.objects.create(...) writes a new OTP record to the database.

The VerifyOTPSerializer creates JWT access and refresh tokens after the OTP is confirmed.

```
refresh = RefreshToken.for_user(user)
return {
    'user': {...},
    'refresh': str(refresh),
    'access': str(refresh.access_token),
}
```

The view layer exposes the endpoints. RegisterView uses CreateAPIView, which handles POST creation for you. LoginView and VerifyOTPView use APIView because the login logic is custom.

```
class RegisterView(generics.CreateAPIView):
    queryset = User.objects.all()
    serializer_class = RegisterSerializer
```

- CreateAPIView is a DRF generic view that already knows how to handle a POST request for creating a resource.
- queryset tells DRF which database rows are available as the base data.
- serializer_class tells DRF which serializer should validate and create the object.

The LoginView.post method collects the request data, validates it, creates an ActivityLog, and returns a response. The authentication_classes = [] line means this endpoint does not require a JWT token to be invoked.

The LogoutView uses permissions.IsAuthenticated, which means only logged-in users may call it.

```
class LogoutView(APIView):
    permission_classes = [permissions.IsAuthenticated]
```

The users/urls.py file maps routes to views.

```
urlpatterns = [
    path("register/", RegisterView.as_view(), name="register"),
    path("login/", LoginView.as_view(), name="login"),
    path("verify-otp/", VerifyOTPView.as_view(), name="verify-otp"),
]
```

The authentication flow is:

- A client sends username and password to /api/login/.

- Django authenticates the user.
- A one-time password is generated and stored.
- An email is sent if possible.
- The client sends the username and OTP to /api/verify-otp/.
- Django validates the OTP, deletes it, and returns JWT tokens.

JWT note

JWT stands for JSON Web Token. It is a signed token that proves a client is authenticated. In this project, the access token lets the client reach protected endpoints, while the refresh token is used to mint new access tokens.

Likely SQL

*When the code checks for an existing username, Django may execute SQL similar to SELECT * FROM auth_user WHERE username = %s; When it creates an OTP record, Django may execute INSERT INTO users_otp (...).*

4. The posts app

Post model, serializer, viewset, and router.

The posts app manages travel blog posts. The Post model stores the title, description, image, timestamps, and author.

```
class Post(models.Model):
    title = models.CharField(max_length=200)
    description = models.TextField()
    image = models.ImageField(upload_to='posts/images/', blank=True, null=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    author = models.ForeignKey(User, on_delete=models.CASCADE, related_name='posts')
```

- ImageField stores file uploads and uses the media system.
- auto_now_add=True fills created_at once, when the row is first created.
- auto_now=True updates updated_at every time the row is saved.
- ForeignKey to User means every post belongs to one author.

The serializer exposes the model data to the API in a controlled way.

```
class PostSerializer(serializers.ModelSerializer):
    class Meta:
        model = Post
        fields = ['id', 'title', 'description', 'image', 'created_at', 'updated_at', 'author']
        read_only_fields = ['id', 'created_at', 'updated_at', 'author']
```

The viewset is a DRF class that handles listing, retrieving, creating, updating, and deleting posts. It also supports search and permission rules.

```
class PostViewSet(viewsets.ModelViewSet):
    queryset = Post.objects.all()
    serializer_class = PostSerializer
    parser_classes = (MultiPartParser, FormParser, JSONParser)
```

- ModelViewSet combines multiple common view behaviors into one class.
- parser_classes tell DRF how to parse incoming data, including multipart form-data for image uploads.
- AllowAny is used for read operations, while IsAuthenticated is used for writes.

The perform_create and perform_update methods create an ActivityLog entry after a post is created or modified.

Likely SQL

*A list request may trigger SELECT * FROM posts_post ORDER BY created_at DESC; search may trigger SELECT ... WHERE title ILIKE %s OR description ILIKE %s;*

5. The travel app

Destination model, serializer, viewset, and router.

The travel app manages destinations, including country, city, category, attractions, budget, duration, and image.

```
class Destination(models.Model):
    title = models.CharField(max_length=200)
    description = models.TextField()
    country = models.CharField(max_length=100, blank=True)
    city = models.CharField(max_length=100, blank=True)
```

- blank=True means the field is optional in forms and serializers.
- The model collects destination metadata that a frontend can display in cards or search results.
- The image field uses the same media upload pattern as posts.

The destination viewset behaves almost exactly like the posts viewset, but its search covers extra fields such as country and city.

Likely SQL

For a search query, Django may generate a WHERE clause over title, description, country, city, travel_category, and author username.

6. Configuration and project settings

Why each configuration value exists and how the project is wired together.

The settings file is the central configuration hub. Every framework component reads from it.

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'rest_framework',
    'corsheaders',
    'posts',
    'users',
    'travel',
]
```

- INSTALLED_APPS tells Django which applications should be loaded and which migrations and templates are available.
- The built-in Django apps provide admin, authentication, sessions, messages, and static files.
- rest_framework enables DRF features.
- corsheaders enables Cross-Origin Resource Sharing so a frontend on a different port can talk to the API.

```
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'corsheaders.middleware.CorsMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
```

```
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
```

- Middleware runs around every request and response.
- SecurityMiddleware adds important security headers.
- SessionMiddleware and AuthenticationMiddleware support sessions and logged-in users.
- CsrfViewMiddleware protects forms against cross-site request forgery.
- CorsMiddleware allows the frontend to call the API from a different origin.

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'postgres',
        'USER': 'postgres',
        'PASSWORD': 'NewPassword123',
        'HOST': 'localhost',
        'PORT': '5432',
    }
}
```

This tells Django to use PostgreSQL instead of SQLite. Django models become SQL tables in the PostgreSQL database.

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),
}

SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(days=7),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=30),
}
```

These settings configure JWT authentication for DRF. The access token is long-lived in this project, which is convenient but not always ideal for production.

Media settings are important because both post and destination images are uploaded. MEDIA_ROOT is the folder on disk where files are stored; MEDIA_URL is the web path that points to them.

7. Models, migrations, and ORM

How Django translates Python models into database tables.

Concept: ORM means Object Relational Mapping. It lets Python code talk to databases without writing raw SQL by hand. Django translates Python operations into SQL for you.

The model classes in posts/models.py and travel/models.py describe tables. Every field becomes a database column. Every instance becomes a row.

Example

When you write Post.objects.all(), Django may translate that into SELECT id, title, description, ... FROM posts_post;

Migrations are versioned instructions that tell Django how to change the database schema over time. The migration files in the migrations folders show how the tables evolved.

```
operations = [
```

```
        migrations.CreateModel(
            name='Post',
            fields=[...],
        ),
    ]
```

- A migration records a database change in a file that can be shared with teammates.
- The migration system prevents manual SQL editing for simple schema changes.
- When you run python manage.py migrate, Django applies the pending migrations.

Why migrations matter

Without migrations, your Python models and your real database could drift apart. Migrations keep them aligned.

8. Request lifecycle and endpoint walkthroughs

The full path from browser to response.

Every request follows a common path:

```
Browser/Postman
   ↓
HTTP request
   ↓
Django server
   ↓
URL dispatcher
   ↓
View
   ↓
Serializer / model / DB
   ↓
JSON response
```

The endpoints in this project are:

- POST /api/register/ → RegisterView
- POST /api/login/ → LoginView
- POST /api/verify-otp/ → VerifyOTPView
- POST /api/logout/ → LogoutView
- GET /api/activity-logs/ → ActivityLogListView
- GET/POST/PATCH/DELETE /api/posts/ and /api/posts// → PostViewSet
- GET/POST/PATCH/DELETE /api/destinations/ and /api/destinations// → DestinationViewSet

The register endpoint is a classic create flow. The browser sends a JSON body with username, email, and password. The URL routes to RegisterView. DRF uses the RegisterSerializer to validate input and create a User. The response returns the created user ID and username. The activity log records the registration event.

The login endpoint validates username/password, generates an OTP, and returns a message. The actual authentication occurs in the serializer, not in the URL routing layer.

The verify OTP endpoint is where the JWT tokens are issued. After verification, the client can use the access token for protected routes such as logout and POST/PUT/DELETE operations.

The posts and destinations endpoints are built with ViewSet classes. DRF automatically handles common RESTful actions. A GET request to /api/posts/ goes to list; POST goes to create; GET /api/posts/1/ goes to retrieve; PATCH goes to partial_update; DELETE goes to destroy.

9. Practical debugging guide

What to look at when something breaks.

When you see an error, first identify the stage of the request lifecycle where it failed.

- If the URL does not match, check the app urls.py and project config/urls.py.
- If the view raises an error, inspect the serializer and request body.
- If the database layer fails, check migrations and the database connection settings.
- If the response is wrong, inspect the serializer fields and the view logic.

Common beginner mistakes:

- Forgetting to add an app to INSTALLED_APPS.
- Using the wrong URL prefix such as /posts/ instead of /api/posts/.
- Ignoring permissions and accidentally allowing unauthenticated writes.
- Forgetting that ImageField requires media configuration.
- Expecting the database to update without running migrations.

10. Interview questions, quizzes, and exercises

Use this section to reinforce the concepts.

Interview questions:

- What is the difference between a view and a serializer?
- What does a ForeignKey represent in Django?
- Why do we need migrations?
- What does JWT authentication do?
- What is the purpose of ModelViewSet?

Mini quiz:

- True or false: Django models are the same thing as database tables.
- What happens when you call serializer.is_valid(raise_exception=True)?
- Which folder contains the main project settings?
- What is the purpose of MEDIA_ROOT?

Practice exercises:

- Add a new endpoint that returns all posts sorted by title.
- Create a serializer that hides the author field from public responses.
- Write a custom permission that allows only the author to update a post.
- Add a search filter for destinations by budget.

11. Glossary

Definitions of key Django, DRF, Python, and backend terms.

- Django: a Python web framework for building web applications.

• DRF: Django REST Framework, an extension that helps build APIs.

• Model: the Python class that represents a database table.

• Serializer: a class that validates and transforms data.

• View: Python code that handles a request.

• ORM: Object Relational Mapper, which translates Python to SQL.

• Migration: a versioned change to the database schema.

• JWT: a signed token used for authentication.

• QuerySet: a lazy database query object.

• Middleware: code that runs around every request and response.